The Minute Forecaster (TradingGame v4): a Supervised Transformer for the Next-Minute Return Distribution

Prabal Negi
`market-agent` project
Architecture and evaluation write-up drafted with Claude (Anthropic)

October 2026

Abstract

Version 4 of `TradingGame` drops the reinforcement-learning game and learns the market's short-term dynamics directly. A single transformer, shared by every stock, reads the last M=96 one-minute states of N stocks and predicts, at every minute and for every stock, a *distribution* for the next minute's log-return (a Student-t with a location and a bounded scale), together with the next timestamp. One-minute returns are nearly unpredictable in the mean, so the scale of the distribution is where most of the learnable signal is. This note specifies the data pipeline, every layer with its matrix form and parameter count, the loss (β-weighted negative log-likelihood), the training procedure, and the evaluation protocol, including a linear autoregressive baseline scored by exactly the same code. In one 80-epoch run (59,749 parameters, 80 training and 80 disjoint validation companies, validated on 2025) the network improves the validation likelihood over a constant per-stock scale by 0.140 nats per target and over an EWMA scale by 0.160, reaches a mean-forecast skill of 8.6% over predicting zero, and keeps its 68% and 95% intervals calibrated; the autoregressive baseline gets 0.050 nats and 1.6% skill.

Contents

1 Problem and scope **2**

2 Data **2**
2.1 The minute cache . . . 2
2.2 Time features and the timestamp description . . . 3
2.3 Per-stock context . . . 3
2.4 Windows, targets and masks . . . 3
2.5 Train and validation split . . . 3

3 Architecture **3**
3.1 Building blocks . . . 4
3.2 Layers . . . 4
3.3 Parameters and cost . . . 6

4 Output distribution and loss **6**
4.1 Student-t likelihood 6
4.2 β-weighted likelihood 6
4.3 Total loss 6

5 Training **7**

6 Evaluation **7**
6.1 A linear autoregressive baseline 8

7 Results of one run **8**

8 Limitations and next steps **9**

1 Problem and scope

The v1–v3 game trains a policy by reinforcement learning on 15-minute or hourly bars; after a full v3 run the policy had not learned anything useful. v4 steps back from decisions and asks a prior question: *how much of the market's next minute can be predicted at all?* It strips the input to stock prices (no news, macro series or portfolio state) and makes the task supervised.

Task. For a window of M consecutive trading minutes of N stocks, predict for every stock i and every minute t of the window the distribution of the next minute's return $r_{t+1,i}$, and the description of the next bar's timestamp. The forecast at minute t may use minutes $\leq t$ only (causality).

Design decisions.

- *A distribution, not a point.* A one-minute return is dominated by noise, so a point forecast is almost always close to zero. The model outputs a location μ and a scale σ and is trained by likelihood.
- *Shared weights.* No parameter depends on which company a column holds, so the training and validation companies may differ and so may N.
- *Forecaster first.* A trading policy on top of the forecaster is a later step; this note covers the forecaster only.

2 Data

2.1 The minute cache

`build_minute_cache.jl` reads each symbol's 1-minute OHLCV file and aligns all symbols on one axis of trading days × 375 session minutes (09:15–15:29). A day joins the axis when at least 20% of the symbols have 10 or more session bars in it. A minute with no row keeps the previous close (forward fill) and is flagged *untraded*; a row with volume above zero is *traded*. Only returns and the traded flag are stored; the model never sees a price level. The percent log-return of the close is

$$r_{t,i} \;=\; \mathrm{clip}\Big(100 \ln \frac{c_{t,i}}{c_{t-1,i}},\; \pm 10\Big), \tag{1}$$

and before it enters the network it is divided by s, the root-mean-square return of the traded minutes of the cache ($s \approx 0.32$ for the cache used in Section 7). All distributional quantities below are therefore in units of one typical move, which makes the output scale bounds and the Student-t comparable across caches.

2.2 Time features and the timestamp description

For the bar at minute t the model receives K=5 time features

$$\tau_t = \Big[\tfrac{m_t}{375},\ \sin\tfrac{2\pi w_t}{5},\ \cos\tfrac{2\pi w_t}{5},\ \ln(1+g_t),\ \mathbf{1}[\text{first bar of the day}]\Big]^\top \in \mathbb{R}^5, \tag{2}$$

with m_t the minute of the day, $w_t \in \{0,\dots,4\}$ the weekday and g_t the number of calendar days since the previous bar. The three-number *timestamp description* the model predicts for the next bar is $\big(m_{t+1}/375,\ \ln(1+g_{t+1}),\ w_{t+1}/4\big)$.

2.3 Per-stock context

A stock has no fixed slot in the input, so it is described to the network by two numbers computed from the 375 bars (one trading day) *before* the window starts: the mean absolute return and the traded fraction,

$$c_i = \Big[\tfrac{1}{375}\sum_{t=s-375}^{s-1}|r_{t,i}|,\ \ \tfrac{1}{375}\sum_{t=s-375}^{s-1}\mathbf{1}[\text{traded}_{t,i}]\Big]^\top, \tag{3}$$

for a window starting at bar s. No token therefore sees a bar later than its own minute.

2.4 Windows, targets and masks

A training example is a window of M=96 consecutive bars. Every position t of the window yields a target for every stock, $y_{t,i} = r_{t+1,i}/s$, so one window gives $M \times N$ targets (teacher forcing; the causal mask keeps them honest). A target counts, $m_{t+1,i} = 1$, only if the stock *traded* in minute t+1 and minute t+1 is not the first bar of a day. A forward-filled zero is not information, and the overnight gap is not a one-minute move.

2.5 Train and validation split

Validation differs from training in *both* companies and dates: the training companies and the validation companies are disjoint lists, and every window lies entirely inside its date range (in the run reported here, training 2022-01-03 to 2024-12-30 and validation 2025-01-01 to 2025-12-30). No target is shared between the two. Because market-wide movements are shared across stocks, a validation score on unseen companies but training dates could be flattered by the market itself, so the evaluation reports three sets (Section 6).

3 Architecture

The network (Figure 1) has two attention axes on the $M \times N \times d$ tensor. Stock mixing lets a stock's token use what the rest of the market did in the same minute; the time blocks let it use its own history. Normalisation is pre-norm [Xiong et al., 2020] with LayerNorm [Ba et al., 2016], and the feed-forward network uses the GELU [Hendrycks and Gimpel, 2016]. Attention follows Vaswani et al. [2017].

Table 1: Notation. Default values are those of the run in Section 7.

Symbol	Meaning	Value
N	stocks in a window (any number)	80
M	minutes in a window	96
$d,\ H,\ d_h = d/H$	model width, heads, head size	32, 4, 8
L	causal time blocks	4
K	time features per minute	5
$F = 1 + K + 2$	input features per stock token	8
ν	Student-t degrees of freedom	4

3.1 Building blocks

For $A \in \mathbb{R}^{n\times d}$ (n tokens, one per row), with $\mu = \frac{1}{d} A\mathbf{1}_d$ and $\sigma^2 = \frac{1}{d}(A - \mu\mathbf{1}^\top)^{\odot 2}\mathbf{1}_d$,

$$\mathrm{LN}(A) = \big(A - \mu\mathbf{1}_d^\top\big) \oslash \sqrt{\sigma^2\mathbf{1}_d^\top + \varepsilon} \ \odot\ \mathbf{1}\gamma^\top + \mathbf{1}\beta^\top, \tag{4}$$

$$Q = AW_Q^\top + \mathbf{1}b_Q^\top, \quad K = AW_K^\top + \mathbf{1}b_K^\top, \quad V = AW_V^\top + \mathbf{1}b_V^\top, \tag{5}$$

$$O_h = \mathrm{softmax}\Big(\tfrac{Q_h K_h^\top}{\sqrt{d_h}} + B\Big)V_h, \quad h = 1, \dots, H, \tag{6}$$

$$\mathrm{MHA}(A) = \big[O_1 \ \cdots \ O_H\big]W_O^\top + \mathbf{1}b_O^\top, \tag{7}$$

$$\mathrm{GELU}(x) = \tfrac{1}{2}x\Big(1 + \tanh\big(\sqrt{2/\pi}\,(x + 0.044715\,x^3)\big)\Big), \tag{8}$$

where $Q_h, K_h, V_h \in \mathbb{R}^{n\times d_h}$ are the h-th column blocks, $W_Q, W_K, W_V, W_O \in \mathbb{R}^{d\times d}$, and B is an additive mask ($B = 0$ for stock mixing, lower-triangular $0/-\infty$ for the time blocks).

3.2 Layers

1. Stock embedding. For minute t the token matrix is $X_t = [\, r_t \ \ \mathbf{1}_N\tau_t^\top \ \ C\,] \in \mathbb{R}^{N\times F}$ and

$$E_t = X_t W_e^\top + \mathbf{1}_N b_e^\top \in \mathbb{R}^{N\times d}, \qquad t = 1, \dots, M, \tag{9}$$

with $W_e \in \mathbb{R}^{d\times F}$ shared by every stock and every minute.

2. Stock mixing. Within each minute the N tokens attend to each other with no positional code, so the order of the stocks is irrelevant:

$$E'_t = E_t + \mathrm{MHA}\big(\mathrm{LN}(E_t)\big), \qquad s_t = \tfrac{1}{N} E_t'^\top \mathbf{1}_N \in \mathbb{R}^d. \tag{10}$$

The mean s_t is the market summary of the minute; $S = [s_1 \ \cdots \ s_M]^\top \in \mathbb{R}^{M\times d}$.

3. Fuse and position. For each stock i (with $E'_{\cdot,i} \in \mathbb{R}^{M\times d}$ its tokens over time),

$$Z_i = E'_{\cdot,i} + SW_s^\top + \mathbf{1}_M b_s^\top + P \in \mathbb{R}^{M\times d}, \qquad W_s \in \mathbb{R}^{d\times d},\ P \in \mathbb{R}^{M\times d} \text{ learned.} \tag{11}$$

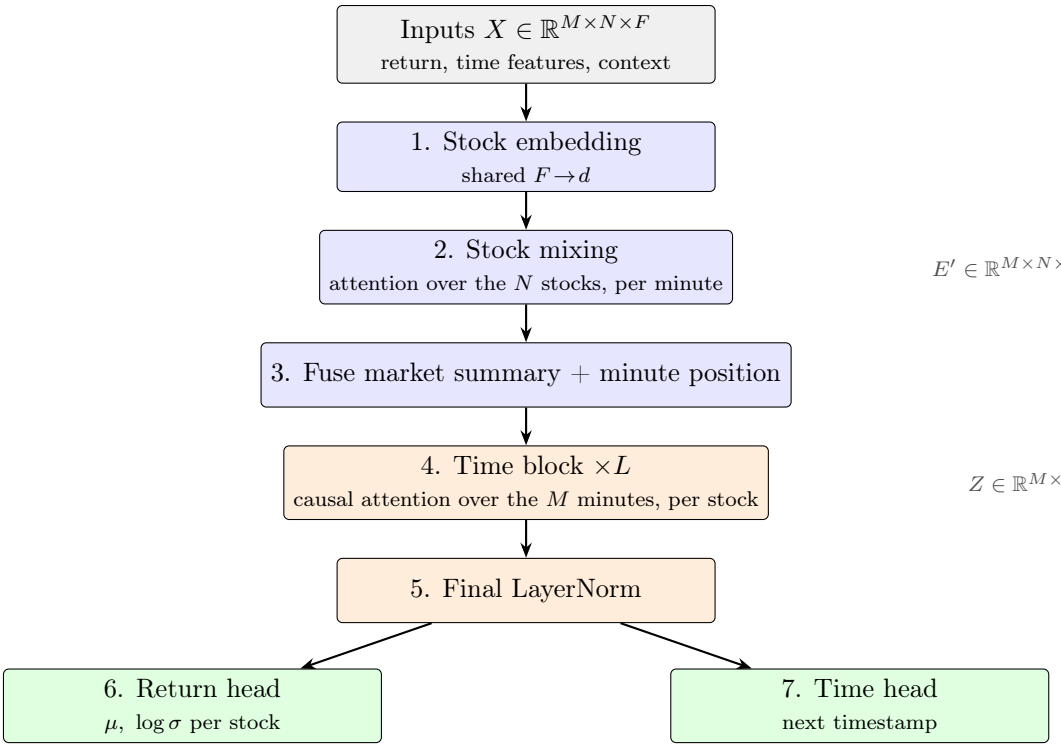


Figure 1: Data flow for one window. Blue layers mix information *across stocks* within one minute; orange layers attend *along time* within one stock.

4. Time blocks (L of them). For one stock, the M tokens are a sequence and minute t may only look at minutes $s \le t$: $B_{ts} = 0$ for $s \le t$ and $-\infty$ otherwise. Each block is

$$Z \leftarrow Z + \mathrm{MHA}\big(\mathrm{LN}(Z)\big), \tag{12}$$

$$Z \leftarrow Z + \Big(\mathrm{GELU}\big(\mathrm{LN}(Z)W_1^\top + \mathbf{1}b_1^\top\big)\Big)W_2^\top + \mathbf{1}b_2^\top, \tag{13}$$

with $W_1 \in \mathbb{R}^{4d\times d}$ and $W_2 \in \mathbb{R}^{d\times 4d}$; every block has its own weights, and the N sequences share them.

5. Final LayerNorm. $H = \mathrm{LN}(Z) \in \mathbb{R}^{M\times d}$ per stock, $H \in \mathbb{R}^{M\times N\times d}$ overall.

6. Return head. One linear map shared by every stock turns each token into a location and a raw scale:

$$\begin{pmatrix}\mu_{t,i}\\ q_{t,i}\end{pmatrix} = W_r h_{t,i} + b_r, \qquad \log\sigma_{t,i} = c + a\tanh q_{t,i}, \qquad r_{t+1,i}/s \;\sim\; t_\nu\big(\mu_{t,i},\sigma_{t,i}\big), \tag{14}$$

with $W_r \in \mathbb{R}^{2\times d}$, $c = -0.5$ and $a = 2.5$, which keeps σ between about 0.05 and 7 return units.

7. Time head. Averaging the tokens of all stocks at a minute, $\bar{h}_t = \frac{1}{N}\sum_i H_{t,i}$,

$$\hat{\tau}_{t+1} = \pi(\tau_t) + D\big(W_\tau \bar{h}_t + b_\tau\big) \in \mathbb{R}^3, \qquad D = \mathrm{diag}\left(\tfrac{1}{375}, 1, 1\right), \tag{15}$$

where $\pi(\tau_t)$ is the calendar's own guess (not learned): one session minute on within a day, and at the last minute of the day the next weekday (with a gap of one day, three after a Friday). Only holidays

defeat this rule, so the head learns a small correction and the minute is exact by construction. W_τ and b_τ are initialised at zero.

3.3 Parameters and cost

Counting weights and biases (LayerNorm has $2d$, the multi-head attention $4(d^2+d)$), the parameters are

$$\underbrace{Fd+d}_{\text{embed}}+\underbrace{2d+4(d^2+d)}_{\text{mixing}}+\underbrace{d^2+d+Md}_{\text{fuse}}+L\,\underbrace{\big(4d+4(d^2+d)+8d^2+5d\big)}_{\text{time block}}+\underbrace{2d}_{\text{final LN}}+\underbrace{2d+2}_{\text{return}}+\underbrace{3d+3}_{\text{time}}. \tag{16}$$

For M=96, L=4, F=8 this gives 59,749 parameters at d=32, 228,037 at d=64 and 890,245 at d=128 (matching the code's count for each). The width d is a hyperparameter to sweep and keep the smallest that matches the best validation result.

The cost is dominated by the attention scores: stock mixing computes $M \cdot H \cdot N^2$ entries and each time block $N \cdot H \cdot M^2$. On a 16 GB card a batch of 32 windows of 80 stocks fits; a batch of 128 does not.

4 Output distribution and loss

4.1 Student-t likelihood

With the location μ and scale σ above, the per-target negative log-likelihood of y under a Student-t with ν degrees of freedom is

$$\ell_\nu(y;\mu,\sigma)=\log\sigma+\tfrac{\nu+1}{2}\log\Big(1+\tfrac{1}{\nu}\big(\tfrac{y-\mu}{\sigma}\big)^2\Big)+c_\nu,\qquad c_\nu=\log\Gamma\big(\tfrac{\nu}{2}\big)-\log\Gamma\big(\tfrac{\nu+1}{2}\big)+\tfrac{1}{2}\log(\nu\pi). \tag{17}$$

Its standard deviation is $\sigma\sqrt{\nu/(\nu-2)}$, so $\nu>2$ is required; the default is $\nu=4$. Minute returns have heavy tails: under a Gaussian, one extreme minute produces a huge gradient and a few outliers can dominate training, whereas the Student-t loss grows only logarithmically in the error. The Gaussian ($\ell(y;\mu,\sigma)=\log\sigma+\frac{1}{2}(\frac{y-\mu}{\sigma})^2+\frac{1}{2}\log 2\pi$) and the Huber loss [Huber, 1964] on the location alone (quadratic below one return unit, linear above; no scale, so no uncertainty) are selectable alternatives.

4.2 β-weighted likelihood

Fitting a location and a scale jointly has a known failure: a model can lower the likelihood by inflating σ where its mean fits badly, so the mean stops improving. Following Seitzer et al. [2022], each term is weighted by $\sigma^{2\beta}$ *held constant* (stop-gradient),

$$\mathcal{L}_{\text{ret}}=\frac{\sum_{t,i} m_{t+1,i}\;\mathrm{sg}\big(\sigma_{t,i}^{2\beta}\big)\,\ell_\nu\big(y_{t,i};\mu_{t,i},\sigma_{t,i}\big)}{\sum_{t,i} m_{t+1,i}},\qquad \beta=0.5, \tag{18}$$

where $\beta=0$ is the plain likelihood. Because the weight changes as σ moves, the *training loss is not comparable across epochs*; evaluation always reports the unweighted likelihood.

4.3 Total loss

The timestamp term is the mean squared error of the description (the minute fraction measured in minutes), $\mathcal{L}_\tau=\frac{1}{M}\sum_t\|\hat\tau_{t+1}-\tau_{t+1}\|^2$, and

$$\mathcal{L}=\mathcal{L}_{\text{ret}}+\lambda_\tau\mathcal{L}_\tau,\qquad \lambda_\tau=0.1. \tag{19}$$

5 Training

- **Batches.** Random window starts into the cache (no file reads in the loop). An "epoch" is a fixed number of such batches (500 steps), not a pass over the data, because windows overlap.
- **Optimiser.** AdamW [Loshchilov and Hutter, 2019] with peak learning rate 3×10^{-4}, weight decay 0.01, a linear warm-up over 200 steps, then a cosine decay to a tenth of the peak [Loshchilov and Hutter, 2017], and gradient-norm clipping at 1.0. Dropout is 0.1.
- **Checkpoint.** After each epoch the three evaluation sets are scored and the checkpoint with the lowest validation negative log-likelihood is kept.
- **Number types.** The forward and backward pass run in `float32` (default), `float16` or `float64`. `float16` keeps float32 master weights, scales the loss dynamically [Micikevicius et al., 2018] (a step whose gradients overflow is skipped and counted), and computes the loss in float32; the checkpoint is always float32. Float8 has no usable Julia type and is not supported.
- **Implementation.** Julia with Flux [Innes, 2018]; the run's configuration is one TOML file read by `scripts/train_minute_forecaster.jl`, and the training page streams its progress.

6 Evaluation

Because one-minute returns are close to unpredictable in the mean, every score is reported next to trivial predictors, on the same masked targets.

Evaluation sets. `val` (validation companies, validation dates; decides the checkpoint), `val_train_companies` (training companies, validation dates) and `train` (training companies, training dates). Comparing `val` to `val_train_companies` isolates the cost of new companies; comparing `val_train_companies` to `train` isolates the cost of new dates.

Location. With μ the predicted location and y the target,

$$\text{skill} = 1 - \frac{\text{MSE}(\mu, y)}{\text{MSE}(0, y)}, \qquad \text{hit rate} = \Pr\left[\operatorname{sign}\mu = \operatorname{sign} y \;\middle|\; y \neq 0,\ m = 1\right]. \tag{20}$$

Skill 0 means no better than predicting no move, and negative means worse; 50% is a coin for the hit rate. The *repeat-last* baseline sets $\mu_{t,i} = r_{t,i}$.

Scale and calibration. The likelihood is scored against two baselines that predict a zero location,

$$\text{gain}_{\text{const}} = \overline{\ell_\nu(y; 0, \sigma_{\text{const}})} - \overline{\ell_\nu(y; \mu, \sigma)}, \qquad \text{gain}_{\text{EWMA}} = \overline{\ell_\nu(y; 0, \sigma_{\text{EWMA}})} - \overline{\ell_\nu(y; \mu, \sigma)}, \tag{21}$$

in nats per target, where σ_{const} is one scale per stock (its RMS return, converted to the distribution's scale) and σ_{EWMA} is the exponentially weighted average of recent squared returns with decay 0.94 per minute [J.P. Morgan/Reuters, 1996]. Both baseline scales are clamped to the range the model's own σ can take, so a stock that never moves cannot win by a scale of zero. A positive gain means the predicted scale carries information beyond the baseline. *Coverage* is the fraction of targets inside the model's own central 68% and 95% interval, $|y - \mu|/\sigma \leq h_p$ with h_p the Student-t quantile; a calibrated model scores about 0.68 and 0.95 (the likelihood is a strictly proper scoring rule [Gneiting

and Raftery, 2007], so it rewards calibration and sharpness together). The report also gives the skill per 15-minute slot of the session and per stock, and the predicted against the realised standard deviation per slot (the predicted value is $\sigma\sqrt{\nu/(\nu-2)}$; heavy tails put the realised RMS above it).

Timestamp. Accuracy of the minute, day-gap and weekday predictions within a small tolerance. It is close to 100% by construction (Section 3) and shows the head works, not that the market is predictable.

6.1 A linear autoregressive baseline

To see how much of the result lagged returns alone give, `scripts/ar_baseline.jl` fits a linear model on the same windows and scores it with the same code. The location is a ridge regression of the next return on the last k=10 returns of the same stock, with one set of coefficients shared by all stocks (lags before the window start are zero, as the network also sees nothing before its window). For the scale, each stock has a level m_i, its mean absolute return over the previous day divided by its traded fraction; the ratio $E|y-\mu|/m_i$ is regressed on the 25 minute-of-day buckets of the session, the k latest $|r|/m_i$ and an exponentially weighted $|r|/m_i$, and the prediction is multiplied by m_i and divided by $c(\nu) = E|t_\nu|$ to obtain σ (for $\nu = 4$, $c = 1$). The model is multiplicative across stocks because volatility is. It has no cross-stock term and no non-linearity.

7 Results of one run

Setup. $d = 32$ (59,749 parameters), $M = 96$, $L = 4$, $H = 4$, batch 32, 80 epochs of 500 steps, float32, $\nu = 4$, $\beta = 0.5$, on an RTX 5060 Ti (0.75 s per step, 8.3 hours). 80 training companies (2022-01-03 to 2024-12-30) and 80 disjoint validation companies (2025), 256 evaluation windows per set. All numbers are from the saved checkpoint of the last epoch (the validation likelihood had plateaued: it improved by 0.0001 from epoch 70 to 80).

Table 2: Network and linear baseline on the three evaluation sets. NLL is in nats per target, in return-scale units, so only the gains are comparable across caches.

Set	Model	NLL	gain const σ	gain EWMA σ	skill	hit rate	95% cov.
`val`	AR(10)	0.7205	0.050	0.071	1.6%	55.4%	93.6%
	transformer	0.6309	0.140	0.160	8.6%	60.3%	95.6%
`val_train_companies`	AR(10)	0.9529	0.038	0.085	2.3%	53.7%	91.7%
	transformer	0.8467	0.145	0.191	10.2%	59.8%	95.3%
`train`	AR(10)	1.0483	0.064	0.092	3.2%	54.6%	92.0%
	transformer	0.9450	0.167	0.196	11.6%	59.9%	95.2%

Reading the table.

- *Scale is the main result.* The network beats a constant per-stock scale by 0.140 nats and an EWMA scale by 0.160 on unseen companies and dates; the linear scale model gets about a third of that.
- *The mean is weak but real.* Skill is 8.6% on validation (the repeat-last baseline is strongly negative: bid–ask bounce makes a minute partly reverse the previous one). The linear model's skill is 1.6% and its location coefficients are the bounce: -0.21 on the latest return, decaying

to about -0.02 by lag 10. Adding lags does not help (30 or 90 lags give the same skill and a worse scale gain).

- *Calibration.* The network's 68% and 95% intervals contain 67.6% and 95.6% of validation targets; the linear model's 95% interval is too narrow (93.6%).
- *Generalisation.* The gap between the training and validation companies on the validation dates is 1.6 skill points and between training and validation dates 1.4 more, so the network is not badly over-fitted.
- *Spread.* On validation only 1 of 71 scored stocks has negative skill (the worst -3.3%, the best about 16%; nine stocks traded too rarely to score); by time of day the skill stays between about 6% and 11%, highest in the first 15 minutes.

8 Limitations and next steps

- **Is it tradeable?** The best per-stock skill is on thinly traded names, where bid–ask bounce and stale quotes are likely the source. A skill that exists only at quoted prices that cannot be traded is not a trading edge; the next test is to check the traded fraction against the per-stock skill and then to simulate execution costs.
- **One run, one seed.** Differences below about 0.2 skill points between epochs or runs are within the noise of 256 windows per set. The width sweep ($d \in \{32, 64, 128\}$, same seed) has not been repeated on the final cache.
- **The baseline is not the strongest linear model.** It uses only a stock's own lags. A baseline with cross-stock or market-wide terms would score higher, so the network's margin is an upper bound on what cross-stock and non-linear structure add.
- **Stock-specific biases shared by both.** The constant-scale baseline uses each stock's RMS measured on the evaluation targets themselves, which is slightly favourable to the baselines; it applies equally to the network and the linear model.
- **Absolute numbers depend on the cache.** The return scale s and the set of days and symbols changed between caches, so likelihood and skill from different caches are not comparable.
- **No decisions yet.** A policy on top of the forecaster (position sizing from μ and σ, costs, risk limits) is a separate, later step.

References

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

Tilmann Gneiting and Adrian E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

Michael Innes. Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, 3(25): 602, 2018.

J.P. Morgan/Reuters. RiskMetrics—technical document. Technical report, J.P. Morgan, 1996.

Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations*, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations*, 2018.

Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks. In *International Conference on Learning Representations*, 2022.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 2017.

Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.